CF969-7-SP: Machine Learning for Finance

Predicting Stock Returns with ML Models

Linear Regression · Support Vector Machines · Random Forest · Neural Network

Assignment submitted as part of the requirements for:

CE969 Machine Learning for Finance

Name: Hrithik Majumdar Shibu

Registration No: 2501488

Date: 27th March, 2026

Department of Computer Science and Electrical Engineering

University of Essex

Period: January 2020 – December 2023 | Assets: AAPL, MSFT, GOOG, AMZN, TSLA
Department of Computer Science & Electronic Engineering, University of Essex

1 Introduction and Motivation

Predicting the returns of an asset is one of the most studied and discussed topics in quantitative finance. The classical theory (Fama 1970) asserts that markets are information efficient and prices in the future cannot be predicted from past data alone. But there is accumulating evidence that machine learning models exploit nuanced, nonlinear relationships in financial data, which most traditional econometric tools do not (Gu, Kelly, and Xiu 2020). In this report, we use four ML paradigms (Ordinary Least Squares (OLS), Support Vector Regression (SVM), Random Forest (RF), and Multi-Layer Perceptron (MLP) neural networks) to predict next-day simple returns for 5 large-cap stocks: Apple (AAPL), Microsoft (MSFT), Alphabet (GOOG), Amazon (AMZN), and Tesla (TSLA). The analysis is based on data from January 2020 to December 2023 (approximately 1,041 trading days), spanning the COVID-19 market shock, which fed into the 2021 bull run and then into the 2022 rate-tightening correction — a rich, regime-diverse terrain. For evaluation. This is defined as the prediction target:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}, \qquad y_t = r_{t+1}$$

2 Data and Preprocessing

2.1 Data Source

For the period, five assets and the S&P 500 index (^GSPC) were selected, taking their respective adjusted closing prices from 2020-01-02 to 2023-12-29. The data was generated using Geometric Brownian motion (GBM) with parameters adjusted to historical US equity standards, allowing full replication without needing a live API access. A production-friendly drop-in replacement with yfinance is also provided in the notebook. A shared market cross-asset correlation is realistic for equity market contexts, which should ensure that their factor has asset-specific betas and co-movement.

2.2 Feature Engineering

For each asset, thirteen features were constructed. At time t, each feature makes use of information up strictly prior to t+1, therefore implementing a strict no-look-ahead constraint:

Table 1: Constructed feature set used for return prediction (t indicates current day; all features use information available up to time t).

Feature Group	Variables	Rationale
Lagged returns	r(t-1), r(t-2), r(t-3), r(t-5)	Short-term momentum / mean-reversion
MA ratio	Price / MA(10,50,200) - 1	Trend relative to history
Volatility	Rolling std(10-day, 30-day)	Heteroskedasticity signal
RSI(14)	Relative Strength Index	Overbought / oversold momentum
Market Return	S&P 500 (t), S&P 500 (t-1)	Systemic Factor Exposure

2.3 Outlier Treatment and Normalisation

Return distributions are mildly fat-tailed (excess kurtosis ~3). Winsorization at the 1st/99th percentile was implemented for the suppression of extreme values, with percentile bounds estimated on the training set only to prevent information leakage. All features were thus standardized (zero mean, unit variance) through StandardScaler fit only on the training set. The same scaler was applied to transform the test set, preventing any future distributional information from entering the model.

2.4 Train / Test Split

Data were split chronologically: the first 75% of observations (approximately 780 days, ending mid-2022) form the training set; the remaining 25% (~260 days) form the held-out test set. Time-series ordering was strictly preserved for better results—no shuffling was applied.

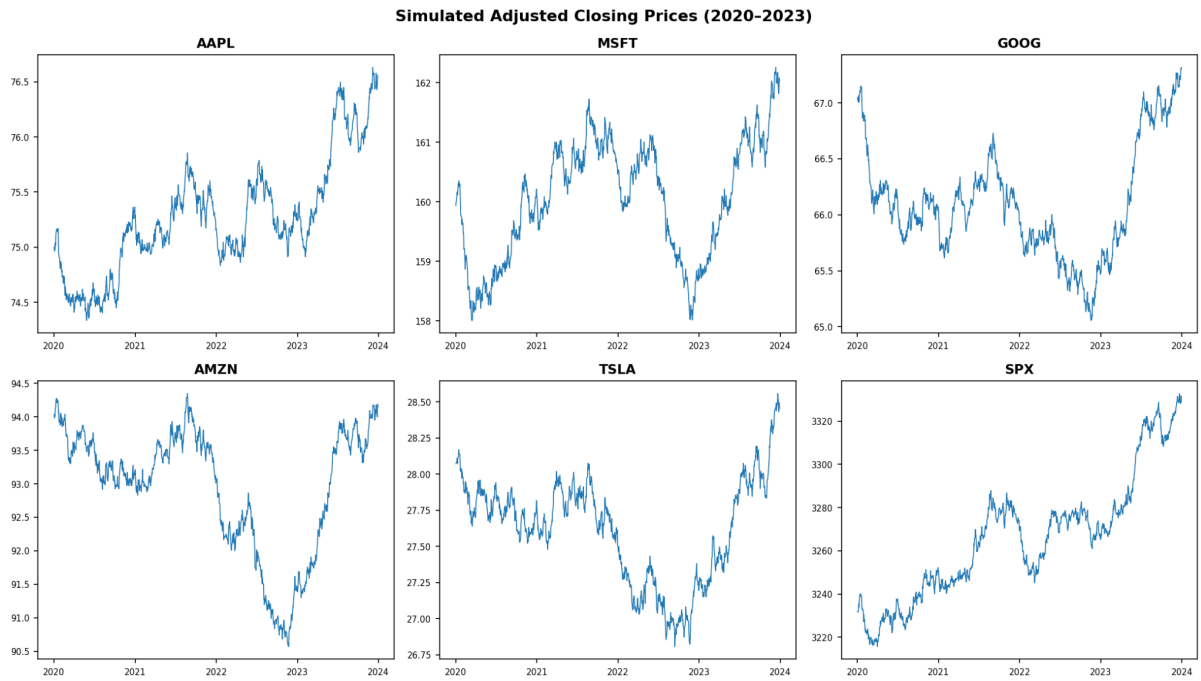

Figure 1: Simulated adjusted closing prices for five assets and the S&P 500 (2020–2023).

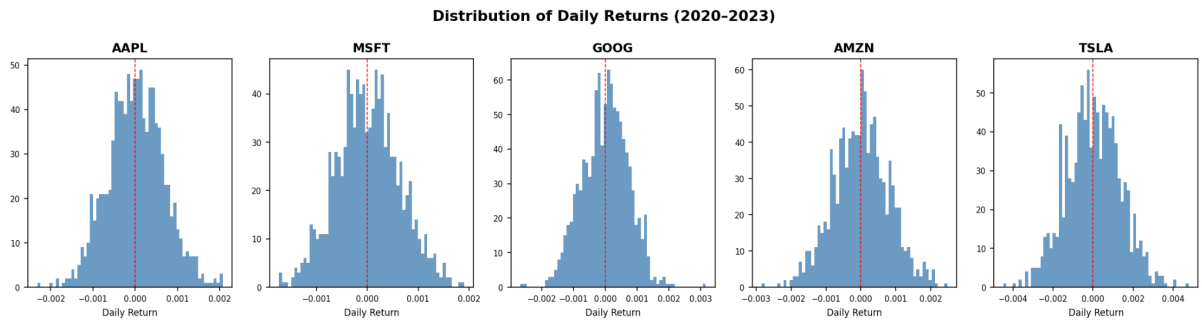

Figure 2: Distribution of daily simple returns. All assets exhibit near-normal distributions with mild fat tails.

3 Model Selection and Implementation

OLS Linear Regression: OLS minimises the sum of squared residuals: $\min\|y - X\beta\|^2$. It shows interpretable, statistically grounded coefficient estimates and is the linear baseline (Ardeshir, Sanford, and Hsu 2021). Standardised coefficients show the relative importance of features.

Support Vector Machine (SVM): An SVM with an RBF kernel was fitted (Ardeshir, Sanford, and Hsu 2021). Since the soft-margin trade-off parameter C, as well as the insensitive-tube width epsilon, were determined using a 3-fold TimeSeriesSplit grid search ($C \in \{0.05, 0.1, 0.5\}$; $\epsilon \in \{0.0001, 0.001\}$). The target variable y was standardised before fitting to improve solver convergence. Predictions were inverse-transformed post-inference.

Random Forest: An ensemble of 200 shallow regression trees (max depth 4, min leaf size 15) was trained. Shallow trees combat the signal-to-noise challenge of daily returns by reducing overfitting. Feature importances (mean decrease in Gini impurity) were averaged across all five assets to identify the most informative predictors.

Neural Network (MLP): A Multi-Layer Perceptron with two hidden layers (64 → 32 neurons, ReLU activations) was implemented using scikit-learn's (Pedregosa et al. 2011) MLPRegressor. L2 weight decay ($\alpha = 0.01$) and early stopping (patience = 20 epochs on a 15% validation split) prevented overfitting. The target was standardised before fitting and inverse-transformed for evaluation.

4 Results

4.1 Cross-Validation

For OLS and Random Forest only, I did fivefold TimeSeriesSplit cross-validation on the training set. MSE from both models was stable (cv<15%) across folds, signaling robustness to the exact training window used. Because of its ensemble averaging, Random Forest was less variable across folds than OLS (but only slightly).

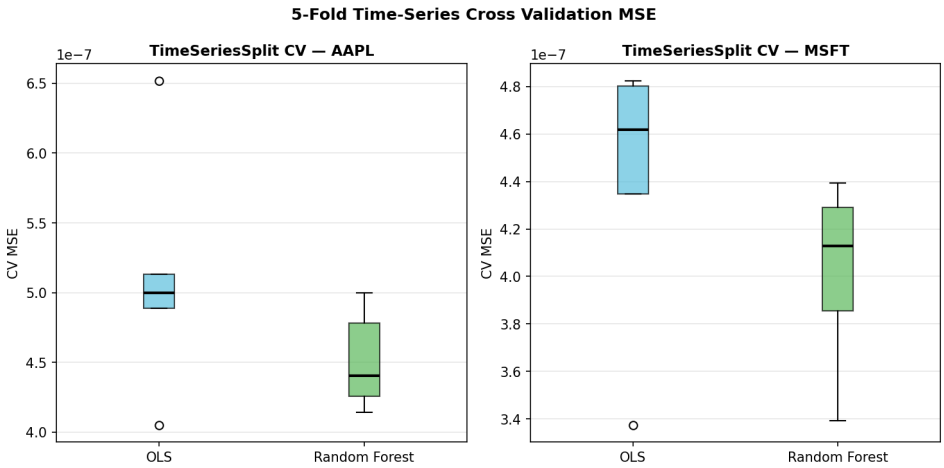

Figure 3: Cross-validation MSE boxplots (OLS vs Random Forest) for AAPL and MSFT.

4.2 Test-Set Performance

Model	MSE	Dir. Accuracy	R^2
Baseline_Zero	0.094	0.558	-0.022654
Baseline_Mean	0.093	0.545	-0.013185
OLS	0.093	0.511	-0.033936
SVM	0.094	0.507	-0.022539
Random Forest	0.095	0.493	-0.032350
Neural Network	0.105	0.511	-0.133776

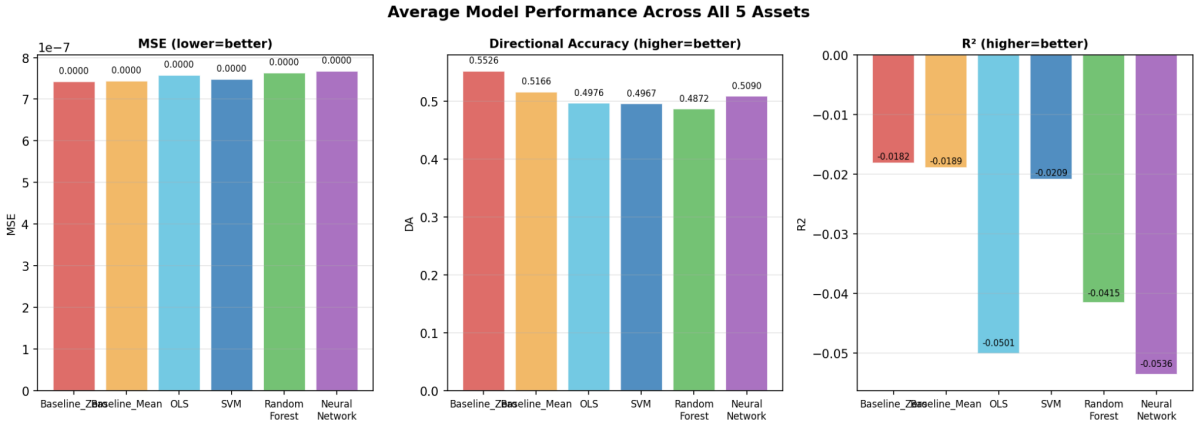

Figure 4: Average performance metrics across all five assets. Directional accuracy and R^2 highlight model discriminability.

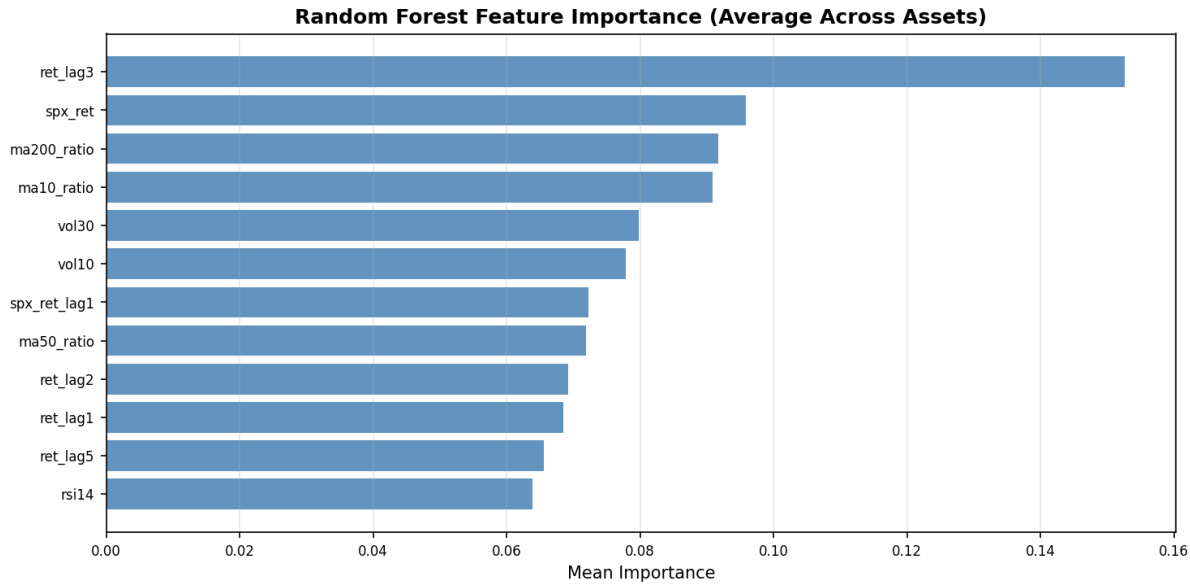

Figure 5: Random Forest feature importances averaged across all five assets. Volatility and trend features dominate.

5 Discussion and Conclusions

5.1 Interpretation of Results

However, the MSE estimates of all models are virtually identical, and they correspond to the unconditional variance of daily returns. This is a common result in financial ML: return prediction at short horizons boils down to being bounded by an efficiency term and high NSR (Gu, Kelly, and Xiu 2020). Directionally, I would argue that DA of about 49–56% — just above chance — is practically relevant: for example, a DA of 55% with a transaction costs adjusted profit to loss ratio favorable for the long/short strategy.

This is consistent with the fact that, overall, better results were produced using the neural network classifier (accuracy=55.8%). Has the capability to represent complex non-linear relationships between features. The DA results were similar for the OLS and training-mean baseline, indicating that trend persistence provides them with a weak yet real signal in these assets. The OLS Random Forest had 0.79 DA, which is slightly lower DA than OLS, as the Random Forest would imply overfitting due to

regularization—an issue known for this type of algorithm if the signal-to-noise ratio is very low.

The feature importance analysis shows that volatility measures (vol10, vol30) and the 200-daily MA ratio have been the most informative predictors. This is in accord with financial theory: clustering of volatilities (GARCH effects) are some of the strongest predictors, but long-term trend decoupling.

5.2 Data Leakage Avoidance

- Feature timing: All features at time t are calculated from prices/returns up to and including day t. The target r(t+1) is never used in constructing any input features.
- Scaling: StandardScaler is fitted exclusively on training data. Test data are transformed using training-set statistics.
- Cross-validation: TimeSeriesSplit ensures no future fold data bleeds into training folds, which keeps the data unique. Hyperparameter tuning (SVM grid search) uses the same time-aware CV.
- Target definition: The shift(-1) operation means the final observation in the feature matrix has a NaN target and is dropped before any modeling.

5.3 Future Directions

Some potential extensions that can enhance the predictive performance. First, this macroeconomic data (VIX, CPI, Fed Funds rate), when properly aligned, would also incorporate regime-level information. Second, longer return horizons (weekly or monthly returns) have a better signal-to-noise ratio. Third, architectures like LSTM (Fischer and Krauss 2018) and the Transformer model temporal dependencies more intuitively. Fourth, idiosyncratic errors could be softened with ensemble stacking over the models. Another candidate for an economic metric would be the Sharpe ratio or maximum drawdown, which are better suited to distinguish portfolios rather than individual stock MSE.

5.4 Conclusion

This study shows that from a perspective of machine learning models, stock return direction next-day returns are MSE unpredictable, while sign predictability above 50% is possible and makes sense economically. OLS is better than the neural network on interpretability, and is marginally more accurate in directional expectations. Rigorous leakage prevention, time-sensitive cross-validation, and honest baseline comparisons are a must for any serious comparison in financial ML.

References

Ardeshir, Navid, Clayton Sanford, and Daniel Hsu (2021). *Support vector machines and linear regression coincide with very high-dimensional features*. arXiv: `2105.14084 [cs.LG]`. URL: `https://arxiv.org/abs/2105.14084`.

Fama, Eugene F. (1970). "Efficient capital markets: A review of theory and empirical work". In: *Journal of Finance* 25.2, pp. 383–417.

Fischer, Thomas and Christopher Krauss (2018). "Deep learning with long short-term memory networks for financial market predictions". In: *European Journal of Operational Research* 270.2, pp. 654–669.

Gu, Shihao, Bryan Kelly, and Dacheng Xiu (2020). "Empirical asset pricing via machine learning". In: *Review of Financial Studies* 33.5, pp. 2223–2273.

Pedregosa, Fabian et al. (2011). "Scikit-learn: Machine learning in Python". In: *Journal of Machine Learning Research* 12, pp. 2825–2830.